

Laboratory work 4

Arrays. Work with one-dimensional arrays

Objective: study of basic algorithms for processing one-dimensional arrays.

Short theoretical information

4.1. The concept of an array.

An array is a structured data type consisting of a fixed number of elements of the same type. The elements of a one-dimensional array are displayed in the form of a table, where $a_0...a_n$ are the elements of the array (Fig. 4.1).

a_0	a_1	a_2	a_3	a_4	a_5	a_6	...	a_n
-------	-------	-------	-------	-------	-------	-------	-----	-------

Fig. 4.1

The number of elements of the array is specified during its description. The general form of declaring a one-dimensional array is as follows:

```
type array_name [size];
```

Example:

```
double value[100];/*declaration of the value array consisting of 100
elements of type double*/
```

The initialization of the array can be carried out during the declaration. The general form of array initialization:

```
type array_name [size] = {list_of_values};
```

Example:

```
int mas[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

```
char str[9] = "alphabet";
```

or

```
char str[9] = { 'a', 'l', 'p', 'h', 'a', 'b', 'e', 't', '\0' };
```

Note. In the C language, the null character ('\0') is the sign of the end of a string, so when declaring a character array in which the string will be stored, its size must be specified by one character more than the expected maximum number of characters.

If the size of the array is not specified when initializing the array, a dimensionless array is created. In this case, the compiler creates an array of such a size that all initializing elements fit into it.

Example:

```
char e[ ] = "Unable to open the file!\n";
```

```
int mas[] = {10, 20, 30};
```

The compiler allocates a block of size $(\text{sizeof}(\text{type}) * \text{array_size})$ bytes under the array in RAM.

Example:

```
int a[10];
```

In the given example, it is $2 * 10 = 20$ bytes.

4.2. References to the elements of a one-dimensional array can generally be represented by indexing, i.e. in the form:

```
array_name[expression];
```

where array_name is a constant pointer;

expression – index, integer type number; it determines the offset – the increase of the address of the given element of the array relative to the address of the zero element of the array.

Example:

```
value[5] = 25.75; /*assigning the element with index 5 to the value 25.75*/
```

A pointer is a variable whose value is an address in memory computer of a certain object (usually another variable). The general form of the index declaration is as follows:

```
type *name;
```

The base type of a pointer is determined by the type of object to which the pointer will refer.

An array name without an index is a pointer to its first element.

Example:

```
int a[10];
```

```
int *p = a; /* r gets the value of a */
```

Here a is a pointer-constant, the address of the beginning of the array, i.e. its zero element, p is a variable (Fig. 4.2); variable p can be assigned a value in one of the following ways:

```
p = a;
```

```
p = &a[0];
```

```
p = &a[i];
```

where $\&a[i] == (a + i)$ is the address of the ith element of the array.

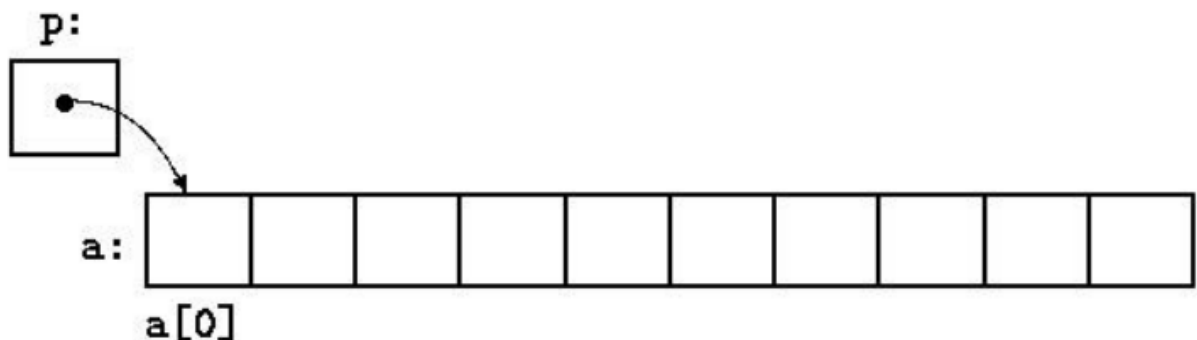


Fig. 4.2

The difference between the variables `p` and `a` is that the value of `a` cannot be changed in the program.

The address of the first element of the array can also be obtained by using the

operator to get the address &. The expressions `a` and `&a[0]` have the same meaning, but professionally written programs do not use expressions of the form `&a[0]`.

According to the type conversion rules of the i -th address value of an array element at the machine level is formed as follows:

`&a[i] = a + i * sizeof(int);`

The following ratios are also valid:

`&a == a + 0 == &a[0]` – the address of `a[0]` – the zero element of the array;

`a + 2 == &a[2]` – the address of `a[2]` – the second element of the array;

`a + i == &a[i]` – the address of `a[i]` is the i -th element of the array;

`*a == *(a + 0) == *(&a[0]) == a[0]` – the value of the 0th element of the array;

`*(a + 2) == a[2]` – the value of `a[2]` – the second element of the array;

`*(a + i) == a[i]` – the value of `a[i]` is the i -th element of the array;

`*a + 2 == a[0] + 2` is the sum of the values of `a[0]` and 2.

If `p` is a pointer to elements of the same type as elements array `a`, and `p = a`, then `a` and `p` are interchangeable, while:

`p == &a[0] == a + 0;`

`p + 2 == &a[2] == a + 2;`

`*(p + 2) == *(&a[2]) == a[2] == p[2];`

`*(p + i) == *(&a[i]) == a[i] == p[i];`

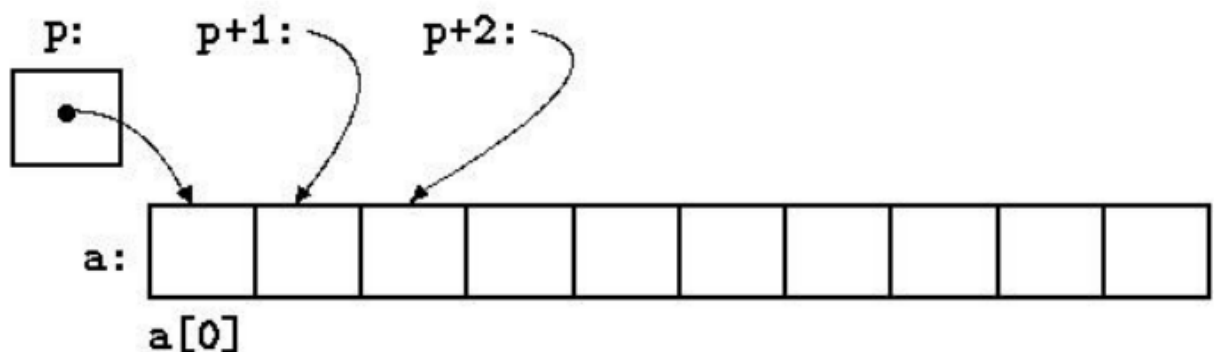


Fig. 4.3

For `a` and `p`, all references to the elements of `a` are equivalent in the form:

`a[i]`, `*(a+i)`, `*(i+a)`, `i[a]`

`p[i]`, `*(p+i)`, `*(i+p)`, `i[p]`

In the following fragment of the program code, the sixth element of the array `i` receives the value 100:

`int *p, i[10];`

`p = i;`

`p[5] = 100; //last two lines are equivalent`

`*(p+5) = 100;`

4.3. Operations on array elements. Actions on elements of the array are performed in the same way as on variables of the corresponding types.

Input-output, as well as processing of one-dimensional arrays, is usually carried out using loops.

Array bounds are not checked when executing a C program. Therefore, the programmer must himself check the compliance with the limits of the array indexes. Going beyond the boundaries of the array leads to corruption of the contents of adjacent memory cells. An example of going beyond the array:

```
double value[100];  
for (int i = 0; i < 110; ++i) value[i] = 0;
```

Example 4.1.

To enter the elements of the array from the keyboard and display them in a line, it is necessary to organize two cycles: in the first cycle, enter all values element by element, in the second - to output each element of the array.

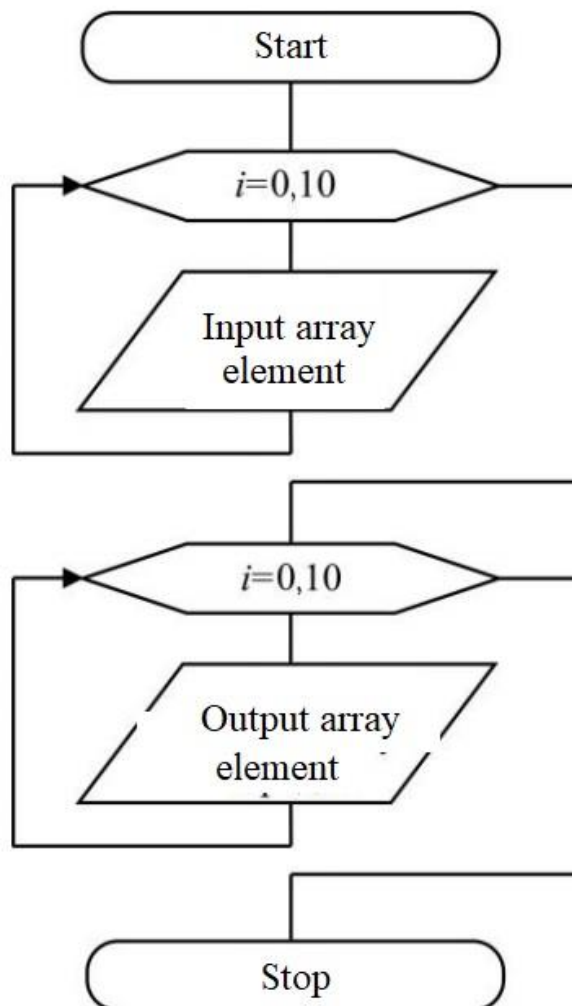


Fig. 4.4 - Block diagram of the algorithm of the program

Program text

```
#define _CRT_SECURE_NO_WARNINGS  
#include <stdio.h>  
#include <locale.h>  
int main(void)  
{  
    int A[10], i;
```

```

setlocale(LC_ALL, "ukr");
printf("Введіть елементи масиву\n");
for (i = 0; i < 10; ++i){
printf("A[%d] = ", i);
scanf("%d", &A[i]);
}
for (i = 0; i < 10; ++i)
printf("%d ", A[i]);
return 0;
}

```

Tasks of laboratory work

Task 1

A sequence of N numbers is given: $a(0), a(1), a(2), \dots, a(N)$.

Find:

- the largest number with an even number;
- the smallest number with an odd number;
- the closest number to the arithmetic mean of these numbers;
- the number of even numbers.

Task 2

1. Given a sequence of N numbers: $a(0), a(1), a(2), \dots, a(N)$.

Find the sum of numbers that have even ordinal numbers and are even.

2. Given a sequence of N numbers: $a(0), a(1), a(2), \dots, a(N)$.

Find the difference between the product of positive and the product of negative numbers.

3. Given two sequences of N numbers: $a(0), a(1), a(2), \dots, a(N)$

and $b(0), b(1), b(2), \dots, b(N)$. Create a new sequence c, forming it from even elements of the sequence a and odd elements of the sequence b, which are multiples of 3.

4. Given a sequence of N numbers: $a(0), a(1), a(2), \dots, a(N)$.

Form a new sequence b, in which the elements will go in a different order and will be paired.

Task 3.

1. Given a sequence of N numbers: $a(0), a(1), a(2), \dots, a(N)$.

Find the sum of numbers that have even ordinal numbers and are even.

2. Given a sequence of N numbers: $a(0), a(1), a(2), \dots, a(N)$.

Find the difference between the product of positive and the product of negative numbers.

3. Given two sequences of N numbers: $a(0), a(1), a(2), \dots, a(N)$ and $b(0), b(1), b(2), \dots, b(N)$. Create a new sequence c, forming it from even elements of the sequence a and odd elements of the sequence b, which are multiples of 3.

4. Given a sequence of N numbers: $a(0), a(1), a(2), \dots, a(N)$.

Form a new sequence b , in which the elements will go in a different order and will be paired.